CHEN PENGXU / LEARNING PORTFOLIO

产品、数据与游戏数值作品集

陈鹏旭 · 完整文档版 · 2026.10

方向	完整收录内容
产品设计	PRD 原文、8 项指标字典、3 类异常、12 条设计验收与实际验证边界
SQL 分析	完整报告、两幅原始图表、16 项核对记录、清洗准备与 8 组查询源码
游戏数值	完整策划案、20 级配置、3 情景、27 组敏感度与 8 条可执行测试

作品性质与阅读边界

三套作品均为 2026-10-08 本轮新生成的 AI 辅助学习材料。业务角色、用户、订单和游戏参数为模拟或自定假设，没有真实企业数据、真实访谈或商业上线成果。本文保留方案、计算与实际验证范围，便于复核；个人贡献以本人实际理解、修改和复现的内容为准。

阅读建议：招聘初筛可先看 3 页速览版；本册用于继续阅读需求规则、分析口径、数值取舍及源码。原型与复现脚本见末尾附件索引。

目录与阅读导航

目录可点击跳转。各章保留原文逻辑与未验证范围，数字均属于学习案例。

指标管理平台 3

实际验证记录 9

模拟用户行为分析 10

16 项实际 SQL / Python 校验 13

查询附录：清洗准备与 8 组 SQL 14

星灯远征：成长数值 19

8 条可执行测试：动作、预期与结果 23

附件与复现索引 24

附件结构：product-design / sql-analysis / game-numerics。可操作原型、数据、源码与 CSV 在素材包中；本 PDF 用于阅读完整文档，不替代这些可复现源文件。

01 / 产品设计

指标管理平台

从人群、时间窗与数据来源出发，定义可以解释的指标。

性质：AI 辅助产品设计练习 | 版本：0.1 | 日期：2026-10-08

本轮新创的模拟案例。所有业务、角色、样本数、负责人均是学习设定；没有开展真实访谈，没有企业在岗经历、正式开发团队或上线成果。文档提出的需求不是已投入生产的功能。

1. 问题与用户场景

假设一家提供付费订阅的在线学习产品，需要统一增长、交易和内容团队的指标。不同角色在报表中都写“付费转化率”，实际可能分别使用注册用户、活跃用户或首购用户作为分母。相同名称并不保证相同统计对象。

模拟角色	场景	当前风险	本练习提出的需求
产品/增长同学	比较新用户注册当天是否完成首购	把旧用户首购混入新注册用户转化	在指标名称和分子分母中明示人群、时间窗
运营同学	搜索“付费”、查看负责人和能否复用	只见数字，不知道来源、成熟时间和版本	可筛选目录，详情列出来源与口径
数据同学	修改定义后检查影响范围	更新分母但下游公式仍引用旧版本	展示依赖/下游关系，版本不一致时阻断最终展示

场景是设计假设，不以“访谈发现”表述。上线前若要验证，应向目标用户核实他们真实的搜索方式、争议口径与使用频率；本轮没有执行这些研究。

2. 目标、非目标与验证计划

目标：让使用者从目录找到正确指标、解释统计口径、识别不可比较的付费率，并在数据异常时看到明确状态和处理动作。

本轮可验收交付：一份需求文档、8 项完整指标定义、本地可操作原型、3 类异常处理与 12 条验收用例。

不做：真实数据库接入、生产 SQL 调度、企业权限系统、可视化看板、任意公式编辑、审批流后端、真实业务效果评估。

后续验证计划，尚未执行：请同学完成“找到新注册当日付费转化并解释分母”“找到净支付金额来源”“判断某指标为什么不可用”三项任务，记录成功/失败、用时与错误原因，反馈后再修改。没有基线前不承诺搜索效率提升或口径错误下降。

3. 优先级与范围

优先级	需求	理由	本轮状态
P0	按名称、ID、标签、负责人搜索；业务域和类型筛选	先解决找到与辨认指标	本地原型交互
P0	详情列出人群、分子分母、时间窗、来源、版本、更新频率	避免同名不同口径	8 项字典与原型
P0	展示直接上游/下游关系	让使用者看到计算依据和变更影响	预置关系，可点击跳转
P0	明示注册、活跃、首购三类分母的语义差异	核心争议案例	口径对比页
P0	分母为零、延迟、版本冲突的状态与处理建议	阻止异常被当作正常结果	原型状态切换
P1	定义编辑及人工审批、历史版本比较	生产治理需要，超出本次演示范围	仅需求设想
P1	收藏、订阅变更提醒	需要真实使用频率验证	仅需求设想
P2	图形依赖编辑、复杂公式、权限后端	增加实现成本	不在本轮交付

以上 P0 是学习作品范围，不意味着实现了真实数据源或正式研发交付。

4. 数据模型与通用约束

统一时间：北京时间 UTC+8，自然日 D 为 [D 00:00:00, D+1 00:00:00)；订单归支付成功时间，不按创建时间。原型中时间戳使用带 +08:00 的 ISO 字符串。

统一人群 E：users.is_test=false AND users.is_internal=false，通过稳定用户 ID 关联。学习案例暂不覆盖游客、跨端用户合并、跨境时区和历史身份变更；若上线须另定规则。

逻辑表（无真实接入）	最少字段	去重/边界
users	user_id、registered_at、is_test、is_internal	user_id 唯一；用于人群与注册归属
events	event_id、user_id、event_at、event_name	event_id 去重；有效活跃行为为 login、content_view、pay_success
transactions	record_id、order_id、user_id、record_type、occurred_at、amount_fen、status	record_id 去重；支付和退款分别为事件；金额以分为单位

首次支付：对 E 中每名用户，取全部可用历史里 record_type=payment AND status=success AND amount_fen>0 的最早 occurred_at。历史不完整则首购指标不可定稿，不能把窗口内第一笔误当人生第一笔。

退款：不抹掉已发生的首购事实；只按退款成功发生日冲减净支付金额。若业务需要“退款后有效购买用户”应另设指标，不能默默修改本定义。

所有指标元数据：ID、名称、业务域、类型、单位、口径、来源、上游ID、统计周期、刷新要求、负责人角色、版本、状态、使用场景。负责人是角色设定，不是实名组织。

5. 8 项指标数据字典

M01 当日新注册用户数

- 业务域/类型/单位：增长 / 基础 / 人。
- 定义：E 中 registered_at 落在 D 的去重 user_id 数；无分母。
- 来源：users；粒度：自然日；版本：v1.0；负责人角色：增长产品。
- 刷新要求：D+1 09:00 前完成 D 的离线汇总；来源未完整时显示“数据延迟”。
- 用途/依赖：新注册队列规模；为 M04、M05、M06、M08 提供注册人群。

M02 当日活跃用户数

- 业务域/类型/单位：内容 / 基础 / 人。
- 定义：E 中在 D 发生至少一次 login、content_view 或 pay_success 的去重用户数；无分母。
- 来源：users + events；粒度：自然日；版本：v1.0；负责人角色：内容运营。
- 刷新要求：D+1 09:00；事件尚未收齐时不定稿。
- 用途：描述行为活跃规模；不能作为新注册付费转化的分母。

M03 当日新增首购用户数

- 业务域/类型/单位：交易 / 基础 / 人。
- 定义：E 中“全部历史最早成功且正金额的支付时间”落在 D 的去重用户数，包含更早注册用户；无分母。
- 来源：users + transactions；粒度：自然日；版本：v1.0；负责人角色：交易产品。
- 刷新要求：D+1 09:00，依赖历史支付完整；同用户多订单只计一次。
- 用途/依赖：衡量首购规模；为 M04 的注册队列交集提供首购条件。

M04 新注册当日首购用户数

- 业务域/类型/单位：增长 / 派生 / 人。
- 定义：同时满足 M01 的注册条件和 M03 的首购条件的去重用户数；无分母。
- 依赖/来源：M01 + M03；users + transactions；版本：v1.0；负责人角色：增长产品。
- 粒度/刷新：自然日，D+1 09:00 且上游完整。
- 用途：把新增注册与新增首购取交集；不直接使用所有当日首购人数。

M05 新注册当日付费转化率

- 业务域/类型/单位：增长 / 复合 / %。
- 分子：M04；分母：M01；公式：M04 / M01 × 100%，分母为 0 返回空值并显示“无可计算样本”，不返回 0%。
- 依赖/来源：M04 + M01；users + transactions；版本：v1.0；负责人角色：增长产品。
- 粒度/刷新：注册队列 D；D+1 09:00 后上游齐备才最终展示。
- 用途：衡量注册当天的首次付费转化；不代表后续累计转化。

M06 新注册次日留存率

- 业务域/类型/单位：增长 / 复合 / %。
- 分子：M01 的 D 注册队列中在 D+1 有 M02 所定义有效活跃行为的去重用户数；分母：M01(D)。
- 来源/依赖：users + events；M01 与 M02 的活跃规则。版本：v1.0；负责人角色：增长产品。
- 公式：分子 / M01(D)；分母为 0 时空值。
- 粒度/成熟：注册队列 D；D+2 09:00 后完整观察与数据汇总才定稿。
- 用途：评价首次使用后翌日再次行为；不使用 D+1 全体活跃人数作分子。

M07 当日净支付金额

- 业务域/类型/单位：交易 / 派生 / 元。
- 定义：E 中 D 发生的成功正金额 payment 之和，减去 D 发生的成功 refund 金额之和。退款可以对应往日订单；因此当日净额允许为负。
- 来源：users + transactions；record_id 去重；支付/退款实际发生时间归日。版本：v1.0；负责人角色：交易产品。
- 粒度/刷新：自然日，D+1 09:00；金额从分换算元，展示保留两位小数。
- 用途：支付流水净额；不是财务收入确认、GMV或现金会计报表。

M08 新注册7日首购转化率

- 业务域/类型/单位：增长 / 复合 / %。
- 分子：M01 的 D 注册队列中，全部历史首次支付发生在 [D 00:00, D+7 00:00) 的去重用户数；分母：M01(D)。
- 来源/依赖：users + transactions；M01 和 M03 的全历史首购规则。版本：v1.0；负责人角色：增长产品。
- 成熟：观察完整覆盖注册日及其后 6 个自然日，D+7 09:00 且数据齐备才定稿；之前显示“观察中”，不是最终转化率。
- 公式：分子 / 分母；分母 0 返回空值。
- 用途：与 M05 区分当天与7日转化；两个不同成熟队列不能直接比较。

6. 付费口径争议案例

下表是人为设定的示例数据：某日新注册 1,000 人、活跃 2,000 人、当日新增首购 160 人，其中当日新注册且首购 80 人。

名称	计算	示例值	可以解释什么
新注册当日付费转化率（M05）	当日注册且首购 80 / 当日注册 1,000	8.0%	新注册队列当天首购比例
活跃用户首购占比（对比示意，未列入8项目录）	当日新增首购 160 / 活跃 2,000	8.0%	活跃中的首购比例，需确认首购包含于活跃行为范围
首购中当日新注册占比（对比示意，未列入8项目录）	当日注册且首购 80 / 当日新增首购 160	50.0%	首购用户的注册时间构成

前两项碰巧相等，不能由数值相等推断口径相同。三个比例不应用同一个指标 ID 或互相替代。

决策：增长场景默认使用 M05；页面名称完整展示“新注册”“当日”“首购”，并固定分子/分母与时间窗。若需要其他人群比例，创建新指标，不修改 M05 的分母。

7. 原型与关联

目录：搜索框匹配名称、ID、标签与负责人角色；业务域与类型为 AND 条件；空结果提供清空入口。点击指标查看定义、来源、版本、直接上游及直接下游。

关系：M01、M03 → M04；M01、M04 → M05；M01、M02 → M06；M01、M03 → M08。M07无目录内上游，但保留来源表。M06 对 M02 是“规则依赖”而不是把 M02 的总人数代入分子；M08 对 M03 是首购判定规则依赖，不能累加每日首购总数替代队列计算。

原型关系展示注明“规则依赖”与“数值依赖”；本轮为预置直接关系，不是自动解析任意 SQL，也不做完整递归血缘治理。

8. 3 类异常

类别	触发	展示与产品动作	验收关键
A1 分母为零	M01=0	结果为空，标签“无可计算样本”；说明当天没有符合范围的新注册用户	不能显示 0%、Infinity 或 NaN
A2 源数据延迟	指标已到可定稿时间，但任一上游尚未完成汇总	显示“数据延迟”，不展示本期最终值；显示需要补齐的上游/刷新要求	不能把旧日期数值标成本日有效值
A3 口径版本不一致	指标要求的上游版本与实际版本不同	显示“口径待复核”，阻断正式对比与导出最终值；列出冲突依赖	不能按未知版本继续计算

多个异常并存时先说明版本冲突、再说明数据延迟、最后解释样本为空；一个异常解除不能掩盖其他异常。本地原型一次模拟一种异常，复合状态属于后续实现范围。

9. 12 条明确验收用例

下表是**产品设计验收用例**。表内预期不意味着真实后端全部实现；实际检查范围另见 VALIDATION.md。

ID	操作/给定条件	预期结果
AC01	目录默认打开	展示8项指标，每项有ID/完整名称/域/类型；示例标签可见
AC02	搜索“付费”	返回 M05、M08，不混入名称不匹配且无同标签的指标
AC03	选择增长域 + 复合类型	返回 M05、M06、M08；两条件同时生效
AC04	搜索“交易产品”	返回负责人角色匹配的 M03、M07；空查询恢复全部
AC05	点击 M05	详情显示 M04、M01 上游；可点入 M01 并查看下游含 M04/M05/M06/M08
AC06	示例新注册1000、活跃2000、首购160、当日注册且首购80	三种对比显示8%、8%、50%；名称、分母与解释均不同
AC07	分母0	空值，“无可计算样本”；没有 Infinity、NaN 或0%误导
AC08	一笔支付在 D+1 00:00（UTC+8）	归入D+1，不计入D；下界00:00计入当日

ID	操作/给定条件	预期结果
AC09	同支付record_id重复出现；同用户多笔成功订单	金额去重；首购取全历史最早成功支付，每用户仅一个首购时间；退款不改变首购时间
AC10	D注册队列未走完7日观察窗；走完且汇总完成	前者显示“观察中”，不定稿；后者按D队列统计，可展示最终值
AC11	选择数据延迟状态	不展示示例最终值；显示待补齐来源、刷新要求和处理动作
AC12	选择版本冲突状态	不展示示例最终值；显示预期/实际版本与“先核对再定稿”动作

10. 发布与面试使用条件

文档/原型是本轮产出，可展示需求推理与学习过程。作者需亲自跑一遍AC01-AC12，逐条检查定义能否被自己解释，并修改不认可的取舍。未经真实业务验证，简历只描述学习交付数量，不能声称口径错误下降、效率提高、真实上线或完成企业指标治理。

实际验证记录

日期：2026-10-08 | 性质：本地学习原型 | AI 辅助

已执行

在工作目录运行 node work/check_metric_lab.js，退出码为 0。读取真实交付文件 model.js，实际执行 12 组规则检查，并解析 HTML 内联 JavaScript 的语法。

检查对应项	实际运行范围	结果
AC01	8项ID唯一、必需元数据字段完整	通过
AC02	搜索“付费”返回M05、M08	通过
AC03	增长域+复合类型返回M05、M06、M08	通过
AC04	负责人查询、空查询、无结果查询	通过
AC05	M05直接上游、M01直接下游的预置关系	通过
AC06	三种示例比例的数值8%、8%、50%	通过
AC07	零分母返回null，异常显示定义为空值符号	通过
AC08	UTC+8自然日上下界，次日00:00不计当日	通过
AC09	支付record_id去重、全历史最早首购、测试用户排除、退款与负净额	通过
AC10	未成熟不定稿、成熟7日队列值、末端时间排除、空队列	通过
AC11	来源未完整时返回数据延迟/null，状态定义含待补来源	通过
AC12	版本冲突状态定义含期望/实际版本，空值与处理动作	通过

额外检查：HTML引用model.js，AI辅助标注存在，验收清单12项，内联JavaScript语法可解析。

未执行或未实现

- 尝试通过浏览器检查入口查看原型，但当前工具返回无可用浏览器，因此**未完成浏览器点击测试或截图布局验收**。已运行的规则检查不等同于12条产品验收已全部完成。
- 没有连接数据库，没有执行生产SQL、真实支付/退款数据、权限服务、定时刷新或真实业务测试。
- UI异常是预置状态切换，版本冲突没有实现自动探测；关系为静态示例，不解析任意SQL。
- 规则检查使用明确构造的模拟输入，未覆盖缺失字段、数据类型错误、重复ID内容冲突、历史不完整等生产数据防御。

作者复核步骤

双击指标平台原型.html，依次搜索“付费”、选择增长+复合、搜索“交易产品”、点M05和M01，打开付费口径页，切换三类异常。查看是否与PRD一致，并调整任何无法自行解释的定义。只有本人已理解并能讲解后，才宜把本作品作为个人学习交付列入简历。

02 / SQL 分析

模拟用户行为分析

完整报告、原始图表与查询源码：先确定口径，再解释结果。

所有数据均由确定随机种子生成，没有真实企业、用户、收入或投放数据。作品由 AI 辅助创建于 2026-10-08，不是此前完成的实习或商业项目。分析结论没有商业验证。

结果与口径

观察期为 **2026-09-01 至 2026-10-07（37日）**，时间统一按 Asia/Shanghai。截止快照为 2026-10-08 00:00:00，采用左闭右开时间区间。数据设定是一个虚构的 AI 工具产品，包含注册、启动、功能浏览、核心操作、付费墙浏览与订单快照。

指标	SQL实际计算值	样本/定义
有效注册用户	1,176 人	原始1,200人剔除24名测试用户
观察期活跃用户	1,044 人	至少发生一次 launch，按 user_id 去重
清洗后事件	22,334 条	范围过滤、测试/不存在用户排除、event_uuid 去重
平均 DAU	192.97 人/日	37个自然日全部纳入，每日不同活跃用户数之和/37
净付费用户	181 人	至少有一个已支付且扣退款后金额大于0的订单
已支付订单	220 笔	订单快照去重后状态paid，包含后续全额退款订单
正净额订单	205 笔	已支付且 amount_cents-refund_cents>0
注册用户付费转化	15.39%	181/1176；观察期并不等长
活跃用户付费转化	17.34%	181/1044
支付总额	¥15,108.00	去重后已支付订单 amount_cents之和/100
已退款金额	¥1,692.60	截止快照前已确认的refund_cents之和/100
净收入（练习口径）	¥13,415.40	支付总额-已退款，不含失败/待支付；非会计收入
注册用户 ARPU	¥11.41	净收入/1176名有效注册用户
ARPPU	¥74.12	净收入/181名净付费用户
注册队列 D7 留存	34.50%	336/974名成熟注册用户，第7个自然日当天有launch

数据质量处理

- 原始事件 **23,678条**，包含测试用户、越界时间、无对应用户与事件重传。有效范围事件记录为23,212条，删除重传 **878条**，得到22,334个业务事件。原始ID是导入行号，event_uuid才是业务事件键。
- 原始订单快照 **572条**，以order_id为业务键，用ROW_NUMBER保留截止日前最新快照。同金额/用户的不同order_id仍是不同订单，不能误删。
- 全额退款订单 **15笔**不计净付费用户和正净额订单；部分退款 **18笔**按剩余净额计入。仅有全额退款订单的用户不属于净付费用户。
- 10-01至10-07注册的 **202人**尚未获得完整D7观察窗，从留存分母中排除。未到观察窗不等于流失。
- 每日收入按订单创建日归属，并采用截止日退款快照回溯重算。它不是每日退款现金流，不适合替代财务核算。
- 事件和订单均先按用户聚合再关联渠道，避免“一人多事件 × 多订单”的JOIN造成收入重复。

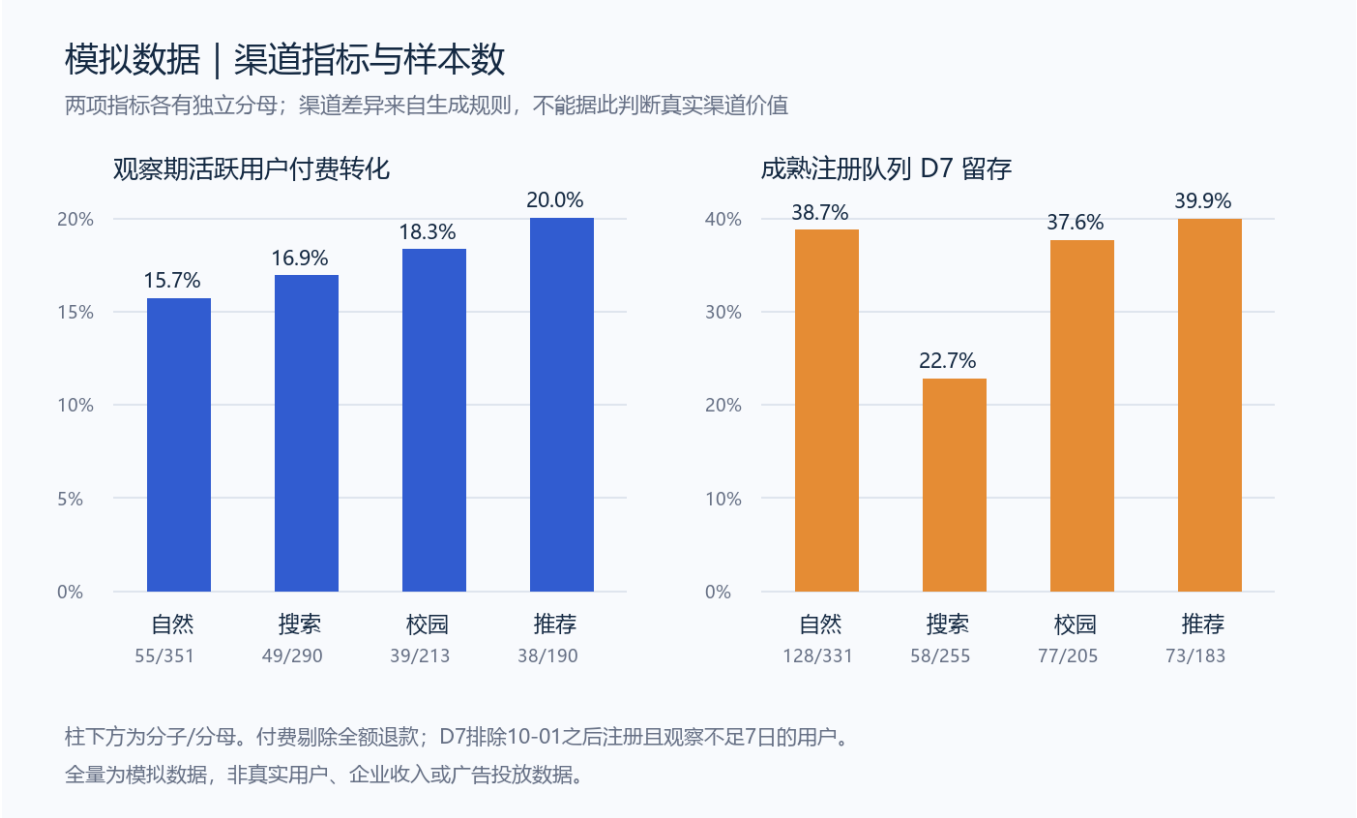
可以解释的发现

1. 同一组净付费用户，以全体有效注册为分母时为 **15.39%**，以观察期活跃用户为分母时为 **17.34%**，相差 **1.95个百分点**。这展示口径差异，不是提升结果。报告应同时写分子、分母、窗口和退款规则。
2. 当前成熟注册队列D7留存为 **336/974=34.50%**。若误把202名未成熟用户当作未留存纳入分母，会变成 **28.57%**，导致低估。不能将两者混用。
3. 模拟渠道中活跃用户付费率最高的是模拟推荐渠道（**38/190=20.00%**），最低的是自然流量（**55/351=15.67%**）。生成器直接设定了渠道活跃差异，同时注册日期分布、抽样噪声和重复访问次数不同，因此不能得出真实渠道优劣或投放建议。

模拟渠道	注册	活跃	净付费	活跃付费率	D7分子/分母	D7留存
自然流量	401	351	55	15.67%	128/331	38.67%
模拟付费搜索	319	290	49	16.90%	58/255	22.75%
模拟校园渠道	243	213	39	18.31%	77/205	37.56%
模拟推荐渠道	213	190	38	20.00%	73/183	39.89%



模拟每日活跃用户（原始模拟图表）



模拟渠道指标与样本数（原始模拟图表）

待验证问题与后续练习

- 如换成经授权的真实数据，需要先核验埋点完整性、业务事件ID稳定性、用户主键、支付和退款时间以及跨时区处理。
- 可新增同龄用户队列付费比较，控制注册时间与观察长度；目前转化是观察期覆盖率，不是每个用户注册后7天转化。
- 如设计实验，需定义主指标/护栏指标、随机分流、样本量和观察期，再分析置信区间。本作品没有开展A/B实验、没有效果提升结论。
- 行为覆盖查询统计“期间至少发生一次”各行为，未施加时序约束，不能称严格顺序漏斗。可另写带事件时间约束的漏斗作为练习。

复现与验证

随机种子：**20261008**。SQLite查询实际执行，生成8份结果CSV。Python从原始字典独立计算事件去重、活跃、付费、金额与D7分子分母，并与SQL对照；另验证渠道/日度金额和总额对账、滚动平均边界。共 **16项**全部通过，见 validation.json。不将校验通过等同于商业结论成立。

此作品可以用于学习和展示SQL实践。本人完成理解、修改及独立复现之前，不宜写成“独立完成”或已掌握所有涉及的技能。

16 项实际 SQL / Python 校验

以下是已有 validation.json 的实际运行记录。所有检查通过仅证明本套模拟输入下的计算一致；没有开展真实业务验证。

编号	实际检查 / 结果	编号	实际检查 / 结果
V01	有效注册人数独立复算 / PASS	V09	D7 留存分子独立复算 / PASS
V02	事件去重独立复算 / PASS	V10	渠道金额与总额对账 / PASS
V03	活跃人数独立复算 / PASS	V11	日度金额与总额对账 / PASS
V04	净付费人数独立复算 / PASS	V12	日度去重活跃对账 / PASS
V05	支付总额独立复算 / PASS	V13	正净额订单与付费用户具有启动事件 / PASS
V06	退款金额独立复算 / PASS	V14	未成熟 D7 用户排除 / PASS
V07	净收入独立复算 / PASS	V15	各比例在 0-1 区间内 / PASS
V08	D7 成熟分母独立复算 / PASS	V16	前六日的完整七日均值不可用 / PASS

查询附录：清洗准备与 8 组 SQL

完整保留 analysis.sql 的清洗准备和八组查询。字体与行宽按 A4 阅读优化，过长行只做视觉折行；复制运行以附件中的原始源码为准。代码作用于模拟 SQLite 数据库。

SETUP / 数据去重、最新快照与日活缓存

```
-- AI工具产品的模拟用户行为分析：仅供SQL练习，非企业或真实用户数据。
-- 数据区间 2026-09-01 至 2026-10-07，时区 Asia/Shanghai。
-- 所有时间采用左闭右开区间 [2026-09-01 00:00:00, 2026-10-08 00:00:00)。
-- SQLite 3.25+。本文件可完整执行；脚本还会将标记后的结果导出CSV。

-- SETUP BEGIN
DROP VIEW IF EXISTS clean_events;
CREATE VIEW clean_events AS
WITH scoped AS (
    SELECT e.*, ROW_NUMBER() OVER (
        PARTITION BY e.event_uuid ORDER BY e.ingest_at DESC, e.raw_id DESC
    ) AS retransmit_rank
    FROM events_raw e JOIN users u ON u.user_id = e.user_id
    WHERE u.is_test = 0 AND e.event_at >= '2026-09-01 00:00:00'
      AND e.event_at < '2026-10-08 00:00:00'
      AND e.ingest_at < '2026-10-08 00:00:00'
)
SELECT raw_id, event_uuid, user_id, event_name, event_at, ingest_at
FROM scoped WHERE retransmit_rank = 1;

DROP VIEW IF EXISTS clean_orders;
CREATE VIEW clean_orders AS
WITH snapshots AS (
    SELECT o.*, ROW_NUMBER() OVER (
        PARTITION BY o.order_id ORDER BY o.updated_at DESC, o.raw_id DESC
    ) AS snapshot_rank
    FROM orders_raw o JOIN users u ON u.user_id = o.user_id
    WHERE u.is_test = 0 AND o.created_at >= '2026-09-01 00:00:00'
      AND o.created_at < '2026-10-08 00:00:00'
      AND o.updated_at < '2026-10-08 00:00:00'
)
SELECT raw_id, order_id, user_id, created_at, updated_at, status,
       amount_cents, refund_cents,
       CASE WHEN status = 'paid' THEN amount_cents - refund_cents ELSE 0 END AS net_cents
FROM snapshots WHERE snapshot_rank = 1;

DROP VIEW IF EXISTS user_activity;
DROP TABLE IF EXISTS user_activity_cache;
-- 只物化一次用户日去重结果，避免D7关联子查询反复执行事件窗口去重。
-- 更新raw数据后需重新运行SETUP区刷新缓存。
CREATE TABLE user_activity_cache AS
SELECT DISTINCT user_id, DATE(event_at) AS activity_date
FROM clean_events WHERE event_name = 'launch';
CREATE UNIQUE INDEX activity_user_day ON user_activity_cache(user_id, activity_date);
CREATE VIEW user_activity AS SELECT user_id, activity_date FROM user_activity_cache;
-- SETUP END
```

01 / 总览指标

01_overview

```
WITH counts AS (
    SELECT
      (SELECT COUNT(*) FROM users WHERE is_test = 0) AS registered_users,
      (SELECT COUNT(DISTINCT user_id) FROM user_activity) AS active_users,
      (SELECT COUNT(*) FROM clean_events) AS clean_event_count,
      (SELECT COUNT(*) FROM clean_orders WHERE status = 'paid') AS paid_orders,
      (SELECT COUNT(*) FROM clean_orders WHERE status = 'paid' AND net_cents > 0) AS net_positive_orders,
      (SELECT COUNT(DISTINCT user_id) FROM clean_orders WHERE status = 'paid' AND net_cents > 0) AS
    net_paying_users,
      (SELECT SUM(amount_cents) FROM clean_orders WHERE status = 'paid') AS gross_cents,
      (SELECT SUM(refund_cents) FROM clean_orders WHERE status = 'paid') AS refund_cents,
      (SELECT SUM(net_cents) FROM clean_orders WHERE status = 'paid') AS net_cents,
      (SELECT COUNT(*) FROM user_activity) AS active_user_days
)
SELECT '2026-09-01' AS period_start, '2026-10-07' AS period_end,
       registered_users, active_users, clean_event_count, paid_orders,
       net_positive_orders, net_paying_users,
       ROUND(gross_cents / 100.0, 2) AS gross_paid_yuan,
       ROUND(refund_cents / 100.0, 2) AS refunded_yuan,
       ROUND(net_cents / 100.0, 2) AS net_revenue_yuan,
       ROUND(1.0 * net_paying_users / NULLIF(registered_users, 0), 6) AS registered_to_pay_rate,
       ROUND(1.0 * net_paying_users / NULLIF(active_users, 0), 6) AS active_to_pay_rate,
       ROUND(net_cents / 100.0 / NULLIF(registered_users, 0), 2) AS arpu_registered_yuan,
       ROUND(net_cents / 100.0 / NULLIF(net_paying_users, 0), 2) AS arppu_yuan,
       ROUND(active_user_days / 37.0, 2) AS average_dau
FROM counts;
```

02 / 日度与滚动指标

02_daily_metrics

```
WITH daily AS (
    SELECT c.day,
      (SELECT COUNT(*) FROM users u WHERE u.is_test = 0 AND DATE(u.signup_at) = c.day) AS new_users,
      (SELECT COUNT(*) FROM user_activity a WHERE a.activity_date = c.day) AS dau,
      (SELECT COUNT(DISTINCT o.user_id) FROM clean_orders o
       WHERE o.status = 'paid' AND o.net_cents > 0 AND DATE(o.created_at) = c.day) AS net_paying_users,
      COALESCE((SELECT SUM(o.net_cents) FROM clean_orders o
       WHERE o.status = 'paid' AND DATE(o.created_at) = c.day), 0) / 100.0 AS net_revenue_yuan
    FROM calendar c
)
SELECT *,
       CASE WHEN ROW_NUMBER() OVER (ORDER BY day) >= 7
            THEN ROUND(AVG(dau) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 2)
       END AS dau_7day_mean,
       ROUND(SUM(net_revenue_yuan) OVER (ORDER BY day ROWS UNBOUNDED PRECEDING), 2) AS
    cumulative_net_revenue_yuan
FROM daily ORDER BY day;
```

03 / 渠道指标

03_channel_metrics

```
-- 先聚合到用户粒度，再JOIN，避免一人多事件×多订单导致收入膨胀。
WITH revenue_by_user AS (
    SELECT user_id, SUM(net_cents) AS net_cents,
           COUNT(CASE WHEN status = 'paid' AND net_cents > 0 THEN 1 END) AS net_orders
    FROM clean_orders GROUP BY user_id
), activity_by_user AS (
    SELECT user_id, COUNT(*) AS active_days FROM user_activity GROUP BY user_id
), grouped AS (
    SELECT u.channel, COUNT(*) AS registered_users,
      SUM(CASE WHEN a.active_days > 0 THEN 1 ELSE 0 END) AS active_users,
      SUM(CASE WHEN r.net_cents > 0 THEN 1 ELSE 0 END) AS net_paying_users,
      SUM(COALESCE(r.net_cents, 0)) AS net_cents
    FROM users u LEFT JOIN revenue_by_user r ON r.user_id = u.user_id
    LEFT JOIN activity_by_user a ON a.user_id = u.user_id
    WHERE u.is_test = 0 GROUP BY u.channel
)
SELECT channel, registered_users, active_users, net_paying_users,
       ROUND(1.0 * net_paying_users / NULLIF(registered_users, 0), 6) AS registered_to_pay_rate,
       ROUND(1.0 * net_paying_users / NULLIF(active_users, 0), 6) AS active_to_pay_rate,
       ROUND(net_cents / 100.0, 2) AS net_revenue_yuan
FROM grouped ORDER BY registered_users DESC, channel;
```

04 / 注册队列 D7 留存

04_retention_cohorts

```
-- D7指注册日+7个自然日“当天launch”，并非第1至7日内任意回访。
-- 只纳入注册日<=09-30的成熟队列；10-01至10-07尚无完整D7观察窗。
WITH mature_users AS (
    SELECT user_id, DATE(signup_at) AS cohort_date FROM users
    WHERE is_test = 0 AND DATE(signup_at) <= '2026-09-30'
)
SELECT m.cohort_date, COUNT(*) AS eligible_users,
       COUNT(a.user_id) AS retained_d7_users,
       ROUND(1.0 * COUNT(a.user_id) / COUNT(*), 6) AS d7_retention_rate
FROM mature_users m LEFT JOIN user_activity a
  ON a.user_id = m.user_id AND a.activity_date = DATE(m.cohort_date, '+7 days')
GROUP BY m.cohort_date ORDER BY m.cohort_date;
```

05 / 渠道 D7 留存

05_retention_channels

```
SELECT u.channel, COUNT(*) AS eligible_users,
       SUM(CASE WHEN EXISTS (
         SELECT 1 FROM user_activity a WHERE a.user_id = u.user_id
           AND a.activity_date = DATE(u.signup_at, '+7 days')
       ) THEN 1 ELSE 0 END) AS retained_d7_users,
       ROUND(1.0 * SUM(CASE WHEN EXISTS (
         SELECT 1 FROM user_activity a WHERE a.user_id = u.user_id
           AND a.activity_date = DATE(u.signup_at, '+7 days')
       ) THEN 1 ELSE 0 END) / COUNT(*), 6) AS d7_retention_rate
FROM users u WHERE u.is_test = 0 AND DATE(u.signup_at) <= '2026-09-30'
GROUP BY u.channel ORDER BY u.channel;
```

06 / 用户行为覆盖

06_behavior_coverage

```
-- 这是观察期内用户行为覆盖，不是严格按时间先后约束的转化漏斗。
WITH flags AS (
    SELECT user_id,
      MAX(event_name = 'launch') AS launched,
      MAX(event_name = 'feature_view') AS viewed,
      MAX(event_name = 'core_action') AS used_core,
      MAX(event_name = 'paywall_view') AS saw_paywall
    FROM clean_events GROUP BY user_id
), coverage AS (
    SELECT 1 AS step_order, 'registered' AS behavior, COUNT(*) AS users
    FROM users WHERE is_test = 0
    UNION ALL SELECT 2, 'launch', SUM(launched) FROM flags
    UNION ALL SELECT 3, 'feature_view', SUM(viewed) FROM flags
    UNION ALL SELECT 4, 'core_action', SUM(used_core) FROM flags
    UNION ALL SELECT 5, 'paywall_view', SUM(saw_paywall) FROM flags
    UNION ALL SELECT 6, 'net_paid', COUNT(DISTINCT user_id)
    FROM clean_orders WHERE status = 'paid' AND net_cents > 0
)
SELECT step_order, behavior, users,
       ROUND(1.0 * users / (SELECT COUNT(*) FROM users WHERE is_test = 0), 6) AS share_of_registered
FROM coverage ORDER BY step_order;
```

07 / 付费用户排序

07_top_payers

```
WITH revenue_by_user AS (
    SELECT user_id, SUM(net_cents) AS net_cents, COUNT(*) AS paid_orders,
           MIN(created_at) AS first_payment_at
    FROM clean_orders WHERE status = 'paid' AND net_cents > 0 GROUP BY user_id
), ranked AS (
    SELECT r.*, u.channel,
      DENSE_RANK() OVER (ORDER BY r.net_cents DESC) AS revenue_rank
    FROM revenue_by_user r JOIN users u ON u.user_id = r.user_id
)
SELECT user_id, channel, revenue_rank, paid_orders,
       ROUND(net_cents / 100.0, 2) AS net_revenue_yuan, first_payment_at
FROM ranked WHERE revenue_rank <= 10 ORDER BY revenue_rank, user_id;
```

08 / 数据质量检查

08_data_quality

```
SELECT 'raw_events' AS check_name, COUNT(*) AS records FROM events_raw
UNION ALL SELECT 'out_of_window_events', COUNT(*) FROM events_raw
 WHERE event_at < '2026-09-01 00:00:00' OR event_at >= '2026-10-08 00:00:00' OR ingest_at >= '2026-10-08
    00:00:00'
UNION ALL SELECT 'unknown_user_events', COUNT(*) FROM events_raw e LEFT JOIN users u ON u.user_id =
    e.user_id
 WHERE u.user_id IS NULL AND e.event_at >= '2026-09-01 00:00:00' AND e.event_at < '2026-10-08 00:00:00'
    AND e.ingest_at < '2026-10-08 00:00:00'
UNION ALL SELECT 'test_user_events', COUNT(*) FROM events_raw e JOIN users u ON u.user_id = e.user_id
 WHERE u.is_test = 1 AND e.event_at >= '2026-09-01 00:00:00' AND e.event_at < '2026-10-08 00:00:00' AND
    e.ingest_at < '2026-10-08 00:00:00'
UNION ALL SELECT 'valid_scope_event_rows', COUNT(*) FROM events_raw e JOIN users u ON u.user_id =
    e.user_id
 WHERE u.is_test = 0 AND e.event_at >= '2026-09-01 00:00:00' AND e.event_at < '2026-10-08 00:00:00' AND
    e.ingest_at < '2026-10-08 00:00:00'
UNION ALL SELECT 'deduplicated_events', COUNT(*) FROM clean_events
UNION ALL SELECT 'raw_order_snapshots', COUNT(*) FROM orders_raw
UNION ALL SELECT 'clean_orders_as_of_cutoff', COUNT(*) FROM clean_orders
UNION ALL SELECT 'fully_refunded_paid_orders', COUNT(*) FROM clean_orders WHERE status = 'paid' AND
    net_cents = 0
UNION ALL SELECT 'partially_refunded_paid_orders', COUNT(*) FROM clean_orders WHERE status = 'paid' AND
    refund_cents > 0 AND net_cents > 0
UNION ALL SELECT 'unmatured_d7_users', COUNT(*) FROM users WHERE is_test = 0 AND DATE(signup_at) >
    '2026-09-30';
```

03 / 游戏数值

星灯远征：成长数值

自定规则、资源账与可复现仿真；明确哪些结论仍需试玩验证。

作品性质：2026-10-08 本轮新生成的 AI 辅助设计练习；所有数值为自定假设，未经过玩家验证，非既有游戏经验，非上线成果。

这是一款虚构单人 RPG 的成长系统纸面原型。程序仿真用于检验经验节奏和金币流转，未制作可玩的游戏客户端，也未开展 Unity 开发、真实玩家测试或上线运营。作者归属应写“AI 辅助练习”，用户理解、调整并复现后再讨论个人贡献。

1. 设计目标与范围

- 建立 1-20 级连续成长与 5/10/15/20 级节点奖励，让前期升级较快、后期所需投入递增。
- 假设标准玩家每天投入 30 分钟、战斗成功率 85%，节奏目标为约 7 天到 10 级、40-50 天到 20 级。这是本练习设定的设计目标，不是用户留存指标。
- 用三种投入情景检查轻度玩家是否持续成长、高投入玩家是否过早触顶，以及金币是否阻断升级。
- 仅包含经验、金币、升级和每日任务。战力、装备、付费、掉落随机性、社交、体力限制及关卡难度均不在本轮模型中，不能由本模型推定留存或付费表现。

2. 核心循环与状态规则

选择远征 → 支付 15 金币整备费 → 进行 5 分钟远征 → 获得胜败奖励 → 每日完成至少 3 次远征领取一次任务奖励 → 按等级依次结算升级 → 获得节点奖励 → 下一日继续。

每日模拟先结算全部远征和每日任务，再尝试自动升级。单次整备费在战斗开始前支付，余额不足则当天剩余远征停止；升级需同时满足经验和金币，金币成本先支付、升后奖励后到账。经验够而金币不够时，保留经验和等级，后续补足金币再升级。一次可连升多级，每级奖励仅领取一次。达到 20 级后不再升级；新增经验记录为 overflow_xp，不转化为金币，避免凭空引入资源来源。

3. 公式、奖励与取整

令当前等级为 L、n=L-1，L∈[1,19]：

- 下一等级所需经验：$E(L)=100+35n+10n^2$。
- 基础升级金币：$U(L)=round5(50+8n+1.5n^2)$。round5 为四舍五入到最接近的 5，恰好居中向上取。
- 到达等级 T≥2 的一次性金币奖励：$R(T)=20+5(T-2)+100\times I(T\in\{5,10,15,20\})$。1 级不重复发升级奖励。
- 敏感度升级成本：$U'(L)=round5(U(L)\times成本倍率)$。经验产出为 floor(当日基础经验×经验倍率)，基础经验包含远征及每日任务。
- 20 级 xp_to_next 和 upgrade_cost_to_next 记 0，语义为封顶不适用，不是可无限免费升级。累计到达经验始终从前一等级推算。

来源或消耗	成功	失败	条件与次数
单次远征经验	90	30	每次 5 分钟
单次远征金币	40	10	完成远征后到账
单次远征整备费	-15	-15	开始前支付
每日任务经验	200	200	当日完成至少 3 次远征，只领一次
每日任务金币	60	60	同上
升级消耗	按配置	按配置	先扣金币与经验，再发升后金币奖励

初始金币 300。没有付费入口、交易转移、无限重置或离线收益。金币余额按“期初 + 远征奖励 + 每日任务奖励 + 等级奖励 - 整备费 - 升级成本”核对。

4. 1-20 级配置表

当前等级	升至下一级经验	到达本级累计经验	升至下一级金币	到达本级金币奖励
1	100	0	50	0
2	145	100	60	20
3	210	245	70	25
4	295	455	90	30
5	400	750	105	135
6	525	1150	130	40
7	670	1675	150	45
8	835	2345	180	50
9	1020	3180	210	55
10	1225	4200	245	160
11	1450	5425	280	65
12	1695	6875	320	70
13	1960	8570	360	75
14	2245	10530	410	80
15	2550	12775	455	185
16	2875	15325	510	90
17	3220	18200	560	95
18	3585	21420	620	100
19	3970	25005	680	105
20	0	28975	0	210

详见 level_config.csv。金币单位为游戏金币、经验单位为 XP。所有参数的唯一编辑入口是 config.json，配置表及结果由脚本重新生成；CSV 是静态计算结果，手动编辑 CSV 不会更新仿真。

5. 三种情景与计算方法

仿真持续 90 天。成功序列是确定性的：累计第 N 次尝试的成功总数为 floor(N×假设成功率)，与上次累计成功数相减得到本次胜败。它控制整体比例以便复现，不模拟真实随机波动、不同玩家分布或连续失败概率。情景同时改变投入时间与假设成功率；它们不是只改变时长的因果实验。

情景	每日分钟	假设成功率	到 10 级天数	到 20 级天数	第 30 日等级	第 90 日金币
轻度投入	15	70%	11	70	14	6170
标准投入	30	85%	7	43	17	12920
高投入	60	95%	4	24	20	27230

达到等级的天数为首次达到该等级的自然日序号，开始日记第 1 天，初始化记第 0 天。CSV not_reached 表示 90 天内未达到，不能当作 0。结果是本套假设下的精确程序输出，不是任何真实游戏的玩家行为。

6. 敏感度对比

三种情景分别计算经验产出倍率 0.8/1.0/1.2 × 升级金币倍率 0.8/1.0/1.2，共 27 组；成功率、时间和其余条件保持各自情景的固定值。完整结果见 sensitivity.csv。下表展示标准投入的九组：

经验倍率	升级金币倍率	到 20 级天数	第 90 日金币	经验够但金币不足的日数
0.8	0.8	53	14005	0
0.8	1.0	53	12920	0
0.8	1.2	53	11835	0
1.0	0.8	43	14005	0
1.0	1.0	43	12920	0
1.0	1.2	43	11835	0
1.2	0.8	36	14005	0
1.2	1.0	36	12920	0
1.2	1.2	36	11835	0

这里的 blocked_days 只记录当日全部收入结算后，经验满足而金币不足的状态；不衡量当天途中一度缺钱的时长。unplayed_runs_due_to_gold 单独记录因整备费不足而停止的次数。两者不可混为同一个卡点指标。

计算结果表明：标准投入、固定基础升级费用时，经验倍率 0.8/1.0/1.2 的封顶时间分别为 53/43/36 天。本轮 27 组均没有日结金币卡点，升级费用在 ±20% 区间内只改变余额，没有改变对应的封顶时间，因此在这些假设下金币门槛较宽松。固定成本倍率时，第 90 日余额不随经验倍率改变：每组届时均已封顶，累计远征收入、任务收入、全部升级费用和等级奖励都一致，而封顶后的金币消耗没有新增。这是模型规则导致的结果，需要下一轮增补消耗或低收益压力情景后再评价经济平衡。

7. 资源风险与优先级

1. **封顶后的金币积累**：封顶后升级消耗归零，远征与任务仍产币。需先设计可选外观或营地消耗并测算，再决定是否调整奖励；不直接把玩家金币清零。
2. **时间和成功率被绑在一起**：高投入组默认战斗更熟练，缩短成长时间的来源不止时长。下一轮先固定 85% 成功率，只改变时间。
3. **每日任务门槛**：15 分钟恰好达到 3 次，10 分钟玩家完全拿不到任务，可能出现收益断崖。应测试分阶段任务奖励。

4. **金币升级卡点**：经验与金币是双重门槛，低成功率、低初始金币或高升级费用可能造成循环中断。必须同时观察可开战余额与待升级状态。
5. **平滑成功序列弱化运气风险**：确定性比例不能证明连续失败时体验良好。下一轮用多个固定随机种子检验下分位数。
6. **体验反馈尚缺**：经验曲线递增只证明数学关系，不能证明重复战斗有趣。需要灰盒试玩，观察单次远征时长和升级时机是否被注意到。
7. **无战力反馈**：角色等级没有改变成功率、时长或掉落，缺少成长带来的能力提升。本轮仅验证资源账，不得据此声称形成完整 RPG 数值体系。
8. **宏观指标不可推定**：模型没有流失、支付、用户样本或实验分组，不应将计算结果写成留存、付费或营收提升。

8. 可执行测试与迭代假设

verify.py 实现 8 条测试，并生成 test_cases.csv 和 verification.json：配置递增与一致性、经验阈值、单次连升、金币不足与恢复、封顶与奖励幂等、每日资源账、情景排序、敏感度与复现一致性。每条包含测试动作、预期结果和执行状态。

下一轮迭代采用以下可证伪假设，均未被本轮验证：

假设	修改 / 验证方法	观察指标与停止条件
分阶段每日任务能减轻 10 分钟玩家的收益断崖	把 3 次领奖改为 1/2/3 次分段，并固定成功率比较	10/15/30 分钟到 10 级天数；若 15 分钟以上组奖励被重复领取，停止合入
升级费用提高 20% 可以减少过量金币而不阻断标准成长	本轮敏感度作为初筛，再加入 50% 成功率与连续失败测试	90 日余额、卡点日数、无法开战次数；若无法开战则重新设计保底来源
高投入玩家需要封顶后的独立目标	增加一次性可选营地升级，避免重复扣同一笔钱	单独列产出/消耗与完成时间；不能挪用本轮结果当真实参与率
等级提升能带来可感知能力收益	灰盒加入一个战力因素，记录试玩者反馈	每 5 级的战斗时长、失败次数、反馈；至少先记录真实样本与口径

9. 使用边界

用户可学习公式、复跑脚本、解释上述取舍，并记录自己的参数调整与理由。个人简历最多描述自己实际完成并能解释的部分；本轮默认归属为 AI 辅助生成练习，不补充不存在的游戏时长、商业策划经历或用户验证成果。

8 条可执行测试：动作、预期与结果

以下完整保留 verification.json 中的八条测试动作、预期及运行状态。验证脚本核对配置、资源账、状态边界与结果复现；不衡量真实体验或商业表现。

测试	动作	预期	实际结果
T01 配置递增与一致性	读取 CSV 并复算 1-20 级累计经验与配置	19 段经验严格递增；20 级封顶；累计到 20 级 28975 XP	PASS / 所有断言通过
T02 经验阈值	分别输入 99/100 XP 与 50 金币	99 XP 不升；100 XP 升 2 级，余 20 金币	PASS / 所有断言通过
T03 单次连升	一次输入升至 5 级所需的累计 XP 与全部升级费用	连续升 4 次；成本、经验与一次性奖励账一致	PASS / 所有断言通过
T04 金币不足与恢复	100 XP 配 49 金币，随后补 1 金币	先保留资源不升；补足后升 2 级	PASS / 所有断言通过
T05 封顶与奖励幂等	19 级输入足量经验与 77 额外 XP，到 20 后再加 90 XP	只发一次 20 级奖励；167 XP 进溢出记录，无第 21 级	PASS / 所有断言通过
T06 每日资源账	核对 270 条逐日记录，重新运行并比较 CSV	金币与经验守恒、余额非负、跨日状态一致、结果可复现	PASS / 所有断言通过
T07 三情景排序	复算轻度/标准/高投入情景与摘要	高投入先封顶；三组无金币卡点或无法开战	PASS / 所有断言通过
T08 敏感度与复现	复算 27 组倍率组合并检查经验倍率方向	CSV 完全一致、余额非负；固定基础成本时经验增加不延迟封顶	PASS / 所有断言通过

04 / 阅读导航

附件与复现索引

报告之外的源文件用于检查交互、数据与参数；更新后应重新验证。

目录	文件	阅读 / 运行说明
product-design	PRD.md / VALIDATION.md	本册完整收录需求与实际验证范围
product-design	指标平台原型.html + model.js	下载到同一目录后浏览器打开 HTML；GitHub 预览不执行交互
sql-analysis	分析报告.md / 分析报告.html	完整报告；HTML 已嵌入图表，可离线阅读
sql-analysis	analysis.sql / generate_and_analyze.py	完整查询源码与模拟数据生成、分析脚本
sql-analysis	data/ / results/ / simulation.sqlite	3 份原始 CSV、8 份结果 CSV 与可查询的模拟数据库
sql-analysis	validation.json / reproducibility_check.json	16 项数值对照记录与 CSV 复现记录
game-numerics	策划案.md / config.json	完整规则与参数编辑入口
game-numerics	simulate.py / verify.py	仿真与八条验证的源脚本
game-numerics	level_config.csv / baseline_daily.csv	20 级配置与三情景 270 条逐日计算结果
game-numerics	scenarios_summary.csv / sensitivity.csv	三情景摘要与 27 组敏感度完整计算结果
game-numerics	test_cases.csv / verification.json	八条测试动作、预期、状态和验证记录
三套目录	README.md	各案例的阅读顺序、运行命令与使用边界

复核建议：先读定义与规则，再运行源脚本核对结果；若更改参数或 SQL，应重新生成结果并运行验证。旧验证记录不能证明新规则已经通过。个人简历只陈述自己实际能解释、修改并复现的部分。

本册覆盖三套本轮 AI 辅助学习作品，没有扩写未经核实的 Demo、Coze、企业实习、游戏开发或真实运营成果。